

RELATÓRIO DE IMPLEMENTAÇÃO, VALIDAÇÃO E HOMOLOGAÇÃO

BANCO DE DADOS E SEGURANÇA DA INFORMAÇÃO

Projeto: Repositório Digital Protegido de Conteúdo Acadêmico

Empresa Parceira	Ideia Space Educação Espacial
Equipe	Squad 22 (Residência Acadêmica / Porto Digital)
Responsáveis pela Camada de Banco de Dados	Victor Emanuel e André Luiz
Documento	Relatório técnico de implementação, validação e homologação — V2.1
Versão	V2.1 — 05/10/2026

Documento técnico de registro e homologação da camada de banco de dados — versão V2.1

1. Contexto do Projeto e Desafio Técnico

O projeto consiste no desenvolvimento de uma plataforma web que atue como repositório centralizado de materiais didáticos — divididos em livros, apostilas em PDF e videoaulas — para professores e alunos da Ideia Space Educação Espacial.

O grande desafio técnico é conciliar o acesso fluido ao conteúdo acadêmico com a proteção rigorosa da propriedade intelectual da instituição. O sistema deve impedir ou dificultar ao máximo downloads não autorizados, cópias brutas e capturas de tela.

Reconhecendo a inviabilidade técnica de impedir 100% dos vazamentos, a arquitetura foi projetada sob dois pilares:

- Dificuldade Máxima de Extração: viewer próprio para documentos PDF e streaming de vídeo sem expor links diretos de arquivos.
- Rastreabilidade Nominal Total: marca d'água dinâmica com identificação do usuário logado e registro contínuo de auditoria no banco de dados.

Este documento registra a jornada completa da camada de banco de dados: da proposta original à implementação da V2, os achados da validação real em PostgreSQL, as correções da V2.1 e a homologação final com testes automatizados com asserções.

2. Decisões Arquiteturais e Tecnologias

A. Banco de Dados Relacional: PostgreSQL 18

- Banco relacional maduro com suporte nativo a Row-Level Security (RLS), transações ACID e particionamento declarativo.
- RLS garante que consultas de um aluno retornem estritamente o conteúdo das turmas com matrícula ativa.
- `uuidv7()` nativo no PostgreSQL 18 gera UUIDs ordenados cronologicamente, aproveitados nos logs de auditoria.
- Auditoria imutável via tabela `log_acesso` particionada por mês (append-only por política RLS).

B. Modelo de Roles e Privilégios (Least Privilege)

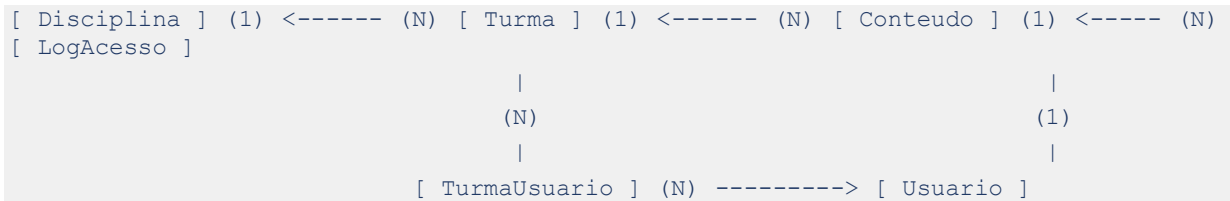
- `app_owner`: role de migração e DDL. Proprietário do schema public. Sem login direto em produção.
- `app_backend`: role de runtime. Sem SUPERUSER, sem BYPASSRLS. Acessa o banco exclusivamente via funções SECURITY DEFINER.
- Superusuário do PostgreSQL: reservado ao DBA para bootstrap e manutenção. Nunca usado pelo backend.

C. Armazenamento de Arquivos: Storage Externo

- Arquivos brutos (PDFs e vídeos) nunca armazenados no PostgreSQL. O banco guarda apenas o `storage_path` interno.
- Constraint `ck_conteudo_storage_path_valido` bloqueia URLs públicas (`http://` e `https://`) no campo de caminho.
- URLs assinadas e temporárias geradas pelo backend sob demanda, após validação do acesso no banco.

3. Modelo de Dados — V2.1

Diagrama Conceitual



Entidades e Atributos Relevantes

Usuario

Armazena identificação, perfil e credenciais. O CPF é armazenado cifrado (cpf_cifrado) e com HMAC para busca (cpf_lookup_hmac), nunca em texto puro. A coluna hash_senha não é acessível ao app_backend via SELECT direto — apenas via fn_obter_hash_senha_por_email.

- Perfis: ALUNO, DOCENTE, ADMIN.
- Status: ATIVA, INATIVA, BLOQUEADA.
- Soft delete LGPD via excluido_em + fn_anonimizar_usuario.

Disciplina / Turma / TurmaUsuario

Hierarquia acadêmica: Disciplina → Turma → matrícula via TurmaUsuario com papel DISCENTE ou DOCENTE e vigência (inicio_em / fim_em). Triggers validam usuário ativo e turma ativa no momento da matrícula.

Conteudo

Metadados de cada material. Tipos: PDF_LIVRO, PDF_APOSTILA, VIDEO_AULA. storage_path aponta para o objeto no storage externo. Índice único parcial em storage_path WHERE excluido_em IS NULL previne duplicatas ativas.

LogAcesso

Tabela particionada por mês (RANGE sobre criado_em). Partições pré-criadas de outubro/2026 a setembro/2027 mais partição DEFAULT. Ações: ABRIU_PDF, INICIOU_VIDEO, ASSISTIU_VIDEO, TENTATIVA_PRINT_DETECTADA, TENTATIVA_ACESSO_NEGADO, GEROU_URL_TEMPORARIA.

4. Estratégia de Proteção de Conteúdo e Segurança

A. Row-Level Security (RLS)

Todas as 6 tabelas têm RLS habilitado. As políticas são aplicadas somente à role app_backend. O superusuário e app_owner não sofrem RLS (sem FORCE ROW LEVEL SECURITY). As políticas garantem:

- Aluno vê apenas conteúdo de turmas com matrícula ATIVA e dentro da vigência.
- Docente vê conteúdo e matrículas da própria turma.
- ADMIN vê tudo; operações de escrita são exclusivas de funções SECURITY DEFINER.
- hash_senha e cpf_cifrado não são colunas expostas via SELECT ao runtime.

B. Funções SECURITY DEFINER

Todo acesso a dados sensíveis e toda escrita são mediados por funções com SECURITY DEFINER. O runtime (app_backend) nunca escreve diretamente nas tabelas; as funções executam com os privilégios de app_owner e aplicam as regras de negócio internamente.

Função	Acessível a	Propósito
app_current_user_id()	app_backend	Lê app.current_user_id da sessão e retorna BIGINT ou NULL.
fn_criar_usuario(...)	app_backend (só ADMIN)	Cria usuário; exige CPF para ALUNO; bloqueia se não for ADMIN.
fn_atualizar_perfil_status_usuario(...)	app_backend (só ADMIN)	Altera perfil e status de conta; restrito a ADMIN.
fn_obter_hash_senha_por_email(email)	app_backend	Retorna hash da senha para autenticação; coluna não exposta diretamente.
fn_obter_dados_watermark(usuario_id)	app_backend	Retorna nome e CPF descriptografado para composição da marca d'água; só próprio usuário ou ADMIN.
fn_atualizar_senha(usuario_id, hash)	app_backend	Atualiza hash da senha do próprio usuário ou de qualquer um se ADMIN.
fn_anonimizar_usuario(usuario_id)	app_backend (só ADMIN)	Soft delete LGPD: zera CPF, desativa conta, preenche excluido_em.
fn_criar_conteudo(...)	app_backend	Insere conteúdo; DOCENTE só na própria turma; ADMIN precisa informar professor_id.
fn_arquivar_conteudo(conteudo_id)	app_backend	Soft delete de conteúdo; só dono ou ADMIN.
fn_registrar_acesso(...)	app_backend	Registra ação no log particionado; valida tipo de material vs. ação.

C. Marca d'Água Dinâmica (Watermark)

A função fn_obter_dados_watermark retorna nome completo e CPF descriptografado do usuário autenticado. O backend injeta esses dados no viewer para compor a marca d'água semi-transparente flutuante. Garante rastreabilidade mesmo em fotos da tela.

D. Viewer PDF, Streaming e Bloqueios de Frontend

- Viewer baseado em Canvas — sem botão de salvar ou imprimir do navegador.
- Vídeos entregues via streaming segmentado com tokens de sessão; sem URLs .mp4 públicas.
- Frontend bloqueia PrintScreen, Ctrl+P, Ctrl+S, F12 e clique direito.
- Conteúdo ocultado quando a aba perde o foco (Page Visibility API).

5. Jornada de Validação: V2 → V2.1

Processo de Validação da V2

Após a entrega da V2, foi realizada uma validação por execução real em PostgreSQL 16 local (ambiente de teste descartável). A migration foi executada na ordem documentada (00 → 01 → 02) e os casos do smoke test foram simulados manualmente.

Resultado: a migration não executou do início ao fim como entregue — quatro erros foram confirmados por execução, não por leitura de código. Dois deles eram bloqueantes de deploy.

Achados Confirmados e Correções Aplicadas na V2.1

#	Achado Original (V2)	Correção Aplicada (V2.1)	Status
1	REVOKE ALL ON ALL TYPES inválido derrubava a migration inteira	Linha removida. Proteção de tipos mantida via ALTER DEFAULT PRIVILEGES; USAGE nos ENUMs concedido explicitamente ao app_backend.	✓Corrigido e testado
2	app_backend sem EXECUTE em app_current_user_id(), travando todos os SELECTs em usuario	GRANT EXECUTE reinserido após o REVOKE ALL ON ALL FUNCTIONS global.	✓Corrigido e testado
3	Anonimização LGPD quebrava para perfil ALUNO (CPF obrigatório mesmo em excluídos)	Constraint ck_usuario_cpf_aluno reescrita para permitir excluido_em IS NOT NULL. fn_anonimizar_usuario seta excluido_em antes de zerar o CPF.	✓Corrigido e testado
4	app_owner sem CREATE efetivo no schema public — migration falhava no primeiro CREATE TYPE	Bootstrap agora faz ALTER SCHEMA public OWNER TO app_owner e concede USAGE, CREATE. Migration valida o owner antes de prosseguir.	✓Corrigido e testado

6. Arquivos Entregues

A V2.1 é composta pelos seguintes arquivos SQL, a serem executados na ordem indicada:

Arquivo	Executado como	Descrição
00_roles_bootstrap_v2_1.sql	Superusuário (DBA)	Cria roles, transfere ownership do schema public para app_owner, concede USAGE e CREATE.
01_v2_1_postgresql18.sql	app_owner (ou membro)	Migration completa: tipos, tabelas, índices, funções, triggers, RLS, privilégios. Requer PostgreSQL 18+.
02_v2_1_smoke_tests.sql	Superusuário (leitura)	Smoke test original: SELECTs de inspeção de catálogo (sem asserções).
02_v2_1_smoke_tests_asserted.sql	Superusuário	Smoke test com asserções (R1–R8). Falha com EXCEPTION em qualquer regressão.
03_v2_1_rls_tests.sql	Superusuário	Testes de isolamento RLS com usuários seed (T1–T17). Tudo em ROLLBACK ao final.

Ordem de Execução

- Como DBA / superusuário: 00_roles_bootstrap_v2_1.sql
- Como app_owner (ou membro): 01_v2_1_postgresql18.sql
- Em banco de testes: 02_v2_1_smoke_tests_asserted.sql e 03_v2_1_rls_tests.sql

7. Resultados dos Testes — PostgreSQL 18

Todos os testes foram executados em PostgreSQL 18 real no Windows 10, banco descartável, com o arquivo original da migration (uuidv7()). Nenhuma adaptação foi necessária.

7.1 Smoke Tests com Asserções (02_v2_1_smoke_tests_asserted.sql)

8 asserções, banco vazio após execução. Cada bloco DO lança EXCEPTION em falha; com ON_ERROR_STOP=1 o psql para no primeiro erro.

Teste	O que valida	Resultado
R1	Migration concluída — 6 tabelas existem	OK
R2	Schema public pertence a app_owner e ele possui CREATE	OK
R3	app_backend executa app_current_user_id()	OK
R4	Definição de ck_usuario_cpf_aluno considera excluido_em	OK
R5	ALUNO anonimizado é aceito pela constraint (ponta a ponta, com ROLLBACK)	OK
R6	Funções de trigger sem EXECUTE direto para app_backend	OK
R7	app_backend sem SUPERUSER nem BYPASSRLS	OK
R8	RLS habilitado nas 6 tabelas	OK

7.2 Testes de Isolamento RLS (03_v2_1_rls_tests.sql)

17 testes com seed de 5 usuários, 2 turmas, 2 conteúdos. Todo o seed é desfeito com ROLLBACK ao final, deixando o banco limpo. Testes executam SET ROLE app_backend com app.current_user_id configurado, simulando o runtime real.

Teste	Ator	Cenário	Resultado
T1	ALUNO A	Vê o conteúdo da própria turma (Turma 1)	OK
T2	ALUNO A	Não vê o conteúdo da Turma 2	OK
T3	ALUNO B	Não vê o conteúdo da Turma 1	OK
T4	DOCENTE C	Vê só o conteúdo da própria turma, não da Turma 2	OK
T5	ADMIN D	Vê conteúdo das duas turmas	OK
T6	ALUNO A	Não enxerga a Turma 2 na tabela turma	OK
T7	ALUNO A	Não enxerga o registro de outro usuário	OK

T8	ALUNO A	hash_senha não é exposto ao app_backend	OK
T9	ALUNO A	INSERT direto em conteudo é negado	OK
T10	ALUNO A	fn_criar_conteudo negada para não-docente	OK
T11	DOCENTE C	fn_criar_conteudo permitida na própria turma	OK
T12	DOCENTE C	fn_criar_conteudo negada em turma alheia	OK
T13	ALUNO B	fn_registrar_acesso negada para conteúdo da Turma 1	OK
T14	ALUNO A	fn_registrar_acesso permitida na própria turma	OK
T15	ALUNO A	fn_criar_usuario negada para não-ADMIN	OK
T16	DOCENTE C	fn_arquivar_conteudo negada para conteúdo de outro docente	OK
T17	ADMIN D	fn_criar_usuario com CPF cria ALUNO corretamente	OK

8. Pendências — Testes após Integração com Backend e Frontend

Os testes das seções anteriores validam a camada de banco de dados de forma isolada. A tabela abaixo lista o que precisa ser testado após a integração com o backend e o frontend, cobrindo os fluxos de ponta a ponta.

Área	O que testar	Critério de aceitação
Backend — Autenticação	Login retorna token; SET LOCAL app.current_user_id é propagado corretamente a cada requisição	SELECT app_current_user_id() retorna o ID correto na conexão do pool
Backend — Criação de usuário	fn_criar_usuario chamada sem contexto de ADMIN	EXCEPTION com SQLSTATE 42501
Backend — Upload de conteúdo	fn_criar_conteudo por DOCENTE em turma própria e em turma alheia	Sucesso na própria; EXCEPTION 42501 na alheia
Backend — Anonimização LGPD	fn_anonimizar_usuario para ALUNO ativo e para DOCENTE	Ambos bem-sucedidos; CPF zerado; excluido_em preenchido
Backend — Senha	fn_atualizar_senha e fn_obter_hash_senha_por_email	Hash atualizado; coluna não exposta ao runtime diretamente
Backend — Watermark	fn_obter_dados_watermark chamada como ALUNO próprio e como outro aluno	Retorna só próprios dados; negado para id alheio sem ser ADMIN
Backend — Arquivamento	fn_arquivar_conteudo por dono e por terceiro	Sucesso para dono/ADMIN; EXCEPTION para terceiro
Backend — Registro de acesso	fn_registrar_acesso com ação ABRIU_PDF em PDF e em VIDEO_AULA	Sucesso no PDF; EXCEPTION 23514 no vídeo

Storage — URLs assinadas	Backend gera URL temporária e banco valida storage_path antes de assinar	URL nunca contém prefixo http:// ou https:// no campo storage_path
Viewer PDF	Abrir PDF via viewer; capturar evidência de que Ctrl+S e clique-direito estão bloqueados	Nenhuma chamada de download direta ao storage; log ABRIU_PDF registrado
Streaming de vídeo	Reproduzir vídeo; inspecionar requisições de rede	Nenhuma URL .mp4 pública; log INICIOU_VIDEO registrado
Frontend — Ocultação de tela	Minimizar janela durante visualização	Conteúdo ocultado; log TENTATIVA_PRINT_DETECTADA se aplicável
Partições de log_acesso	Inserir logs de acesso em outubro/2026 e em outubro/2027	Log de 2026 cai na partição log_acesso_202610; log de 2027+ cai na DEFAULT até nova partição ser criada
Backup e restore	pg_dump do banco migrado; restaurar em banco limpo; rodar os smoke tests	Todos os 8 smoke tests e os 17 testes RLS passam no banco restaurado
Manutenção de partições	Job agendado (pg_cron ou cron externo) criando partições mensais com antecedência	Partição do mês seguinte existe antes do dia 1 do mês
Política de retenção de logs	Definir e documentar prazo de retenção de IP e user-agent (dados pessoais)	Política registrada; job de purga ou anonimização agendado

Observações Importantes

- As funções fn_atualizar_senha, fn_obter_dados_watermark, fn_obter_hash_senha_por_email e fn_atualizar_perfil_status_usuario não foram testadas por asserções no banco isolado — dependem de chamada via backend.
- fn_anonimizar_usuario foi testada por simulação de UPDATE direto (R5). O teste de ponta a ponta via backend deve chamá-la como função e verificar o estado resultante.
- O job de manutenção de partições é crítico: sem ele, logs de meses não pré-particionados caem na partição DEFAULT, e retroparticionamento tem custo elevado.
- A política de retenção de IP e user-agent é exigência de LGPD. Esses dados constam em log_acesso e precisam de prazo de purga ou anonimização documentado e implementado.
- Backup e restore devem ser testados antes do go-live, incluindo a verificação de que os smoke tests passam no banco restaurado.

9. Estado Atual e Próximos Passos

Camada de banco de dados: homologada para integração com o backend.

- Migration V2.1 executada com sucesso no PostgreSQL 18 real (arquivo original com uuidv7()).
- 8 smoke tests com asserções: todos aprovados.
- 17 testes de isolamento RLS: todos aprovados.
- Nenhuma regressão identificada em relação aos quatro achados do relatório de validação.

Próximos passos:

- Integrar o backend à migration V2.1 e executar os testes da Seção 8.
- Configurar o job de manutenção de partições mensais de log_acesso.
- Definir e documentar a política de retenção de dados pessoais nos logs.
- Executar teste de backup e restore antes do go-live.
- Repetir os smoke tests e testes RLS no ambiente de produção após o primeiro deploy.

Ambiente de teste: PostgreSQL 18, Windows 10, banco descartável teste_v21. Nenhum dado real utilizado — apenas usuários fictícios (Aluno A/B, Docente C/E, Admin D) criados para os testes de RLS, todos removidos por ROLLBACK ao final da execução.